SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	S Sandhiya	**Reg. No.:**	192321015
Section / Batch:	Slot D	**Date:**	26-10-2026

Code of Conduct

I S Sandhiya(192321015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S Sandhiya

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.

- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical	Makes reasonable decisions with	Makes weak decisions with	No clear decisions made or

		decisions with strong rationale	some justification	limited justification	rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	**Understanding of Scenario**	**Identification of Key Issues**	**Application of Concepts**	**Decision Making**	**Clarity of Response**	**Sub. Total**	**Total %**
1							
2							
3							
4							
5							

Faculty In-charge	**Course Coordinator**	**Program Director**
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

(a) Greedy Best-First Search: Behaviour, Strengths and Weaknesses

Greedy Best-First Search (GBFS) selects the node that appears closest to the destination based only on the heuristic function \(h(n)\).

\[f(n)=h(n) \]

Here, \(h(n)\) represents the straight-line distance from the drone's current location to the destination.

Behaviour:

- The drone selects the path that appears closest to the hospital or medicine delivery point.
- It uses straight-line distance to choose the next location.
- It can quickly find a route when the environment is simple.
- It may choose a shorter-looking path that is blocked by floodwater or dangerous weather.

Strengths:

- Fast decision-making with low computational effort.
- Useful when an approximate route is needed urgently.
- Easy to implement using distance-based heuristics.

Weaknesses:

- Ignores actual travel costs and risks.

- May select flooded or dangerous routes.
- Does not guarantee the shortest or safest route.
- Partial information can lead to poor decisions.

(b) A* Search: Cost and Heuristic Balance

A* Search considers both the actual cost of reaching a node and the estimated cost of reaching the destination.

\[f(n)=g(n)+h(n) \]

Where:

- \(g(n)\): Actual cost from the starting point to the current node, including travel time and other modeled costs.
- \(h(n)\): Estimated remaining cost to the destination.
- \(f(n)\): Estimated total cost of the route through that node.

Behaviour:

- The drone evaluates both the cost already incurred and the estimated remaining distance.
- It can account for weather, terrain, fuel use, and flood-related delays through \(g(n)\).
- It selects the node with the lowest estimated total cost.
- When a path becomes blocked, the drone can update the route costs and search again.

Advantages:

- Considers actual costs rather than distance alone.
- Can find an optimal route when the cost model and heuristic satisfy the required conditions.
- Better suited to balancing travel time and risk.

Limitation: A* may require more memory and computation than Greedy Best-First Search, especially in a large search space.

(c) Impact of Heuristic Accuracy on Both Algorithms

A heuristic estimates how far or how costly it is to reach the destination.

Heuristic accuracy	**Greedy Best-First Search**	**A* Search**
Accurate heuristic	Usually guides the drone efficiently	Helps find the best route with less unnecessary exploration
Inaccurate heuristic	May select a poor or blocked route	May explore more nodes and become less efficient
Overestimated cost	Can mislead route selection	May lose its optimality guarantee

Underestimated cost	Can still mislead the search	Preserves optimality under standard conditions if the heuristic is admissible and the search is implemented correctly

Example: Straight-line distance may indicate that a destination is nearby, but a flooded river may force a long detour. A heuristic that considers flood conditions and expected delays can provide more useful guidance.

(d) Effect of Dynamic Changes and Blocked Paths

Floods and weather can change the environment while the drone is travelling.

- Path blockage: The drone must remove or avoid inaccessible routes.
- Changing travel costs: Strong winds and heavy rain may increase travel time and risk.
- Repeated search: The algorithm may need to recalculate the route when new information arrives.
- Computational overhead: Frequent changes can increase processing time.
- Safety concerns: Continuing along an outdated route can cause accidents or delay medicine delivery.

To handle these changes, the drone should receive real-time updates, validate route safety, and replan when conditions change. Incremental algorithms such as D Lite* can be considered because they reuse previous search information instead of restarting the entire search after every change.

(e) Recommended Algorithm and Justification

Recommendation: A* Search with real-time replanning and safety constraints.

Criterion	**Greedy Best-First Search**	**A* Search**
Speed	Often faster	May require more computation
Travel cost	Ignores accumulated cost	Considers accumulated cost
Optimality	Not guaranteed	Guaranteed under suitable conditions
Risk management	Requires additional safety checks	Can include modeled risk in route costs
Dynamic environment	Can reselect routes	Can replan using updated costs

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.

- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

(a) Formulate the Problem as an Optimization Problem

The smart city traffic signal system aims to find the best signal timings to reduce waiting time, fuel consumption, and traffic congestion across hundreds of intersections.

Objective function:

$$\min F(x)=w_1W(x)+w_2FC(x)+w_3C(x)$$

Where:

- x = traffic signal timing configuration.
- $W(x)$ = total vehicle waiting time.
- $FC(x)$ = total fuel consumption.
- $C(x)$ = traffic congestion level.
- w_1,w_2,w_3 = weights representing the importance of each objective.

Why traditional search is inefficient:

- Hundreds of intersections create an enormous search space.
- Exhaustive search requires evaluating too many timing combinations.
- Traffic flow changes continuously.
- Signal timings must satisfy safety and minimum/maximum green-light constraints.
- The system needs near-real-time decisions.

Therefore, local search and optimization algorithms are suitable for finding good solutions efficiently.

(b) Hill Climbing Algorithm

Hill Climbing starts with an initial signal timing configuration and repeatedly moves to a neighbouring configuration that improves the objective function.

Working:

1. Initialize signal timings for all intersections.

2. Calculate waiting time, fuel consumption, and congestion.
3. Modify one or more signal timings slightly.
4. Evaluate the new configuration.
5. Accept the change if it improves the objective.
6. Repeat until no better neighbouring solution is found.

Example: If increasing the green-light duration from 30 to 35 seconds reduces waiting time, the system accepts the change.

Advantages:

- Simple to implement.
- Requires relatively little memory.
- Can produce quick improvements.
- Suitable for small, frequent timing adjustments.

Limitations:

- May get stuck in a local optimum.
- Can struggle when neighbouring configurations have similar values.
- May fail to discover a better solution requiring several coordinated changes.
- Needs repeated adaptation as traffic patterns change.

(c) Local Maxima, Plateaus, and Ridges

Although the system minimizes cost, these concepts can be understood using the equivalent landscape of solution quality.

Problem	**Meaning**	**Real-world traffic example**
Local optimum (local maximum in a quality landscape)	A solution better than nearby solutions but not the best overall	A timing plan reduces congestion at one junction but creates delays across the network
Plateau	Many neighbouring solutions have nearly equal quality	Changing a signal by a few seconds produces almost no measurable improvement
Ridge	Improvement requires coordinated changes across several variables	Several connected intersections must adjust their green-light timings together to improve traffic flow

Impact: Hill Climbing may stop prematurely, waste time exploring equivalent configurations, or struggle to coordinate signals across connected intersections.

(d) Simulated Annealing and Genetic Algorithms

1. Simulated Annealing (SA)

Simulated Annealing improves on Hill Climbing by sometimes accepting a worse solution to escape a local optimum.

Working:

- Start with an initial signal timing configuration.
- Generate a neighbouring configuration.
- Accept better solutions immediately.
- Sometimes accept worse solutions with a probability controlled by temperature.
- Gradually reduce the temperature, making acceptance of worse solutions less likely.
- Stop when the time or iteration limit is reached.

The probability of accepting a worse solution is:

\[P=e^{-\Delta E/T} \]

Where \(\Delta E\) is the increase in cost and \(T\) is the temperature.

Advantages: Escapes local optima, uses limited memory, and can explore alternative timing plans.

Limitation: Performance depends on the cooling schedule and parameter selection.

2. Genetic Algorithm (GA)

A Genetic Algorithm uses a population of candidate signal timing configurations inspired by natural selection.

Working:

1. Generate a population of possible signal timing plans.
2. Evaluate each plan using the objective function.
3. Select better-performing plans as parents.
4. Apply crossover to combine their timing configurations.
5. Apply mutation to introduce new timing variations.
6. Repeat until a good solution or stopping condition is reached.

Advantages:

- Explores many candidate solutions simultaneously.
- Can find high-quality solutions in large search spaces.
- Handles coordinated changes across multiple intersections.
- Is less dependent on a single starting solution.

Limitations: Requires more computation, and performance depends on population size, mutation rate, and other parameters.

(e) Recommended Approach

Recommendation: Genetic Algorithm with real-time traffic data and safety constraints.

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm

Escapes local optima	Poor	Good	Good
Search exploration	Limited	Moderate	Broad population-based
Computation	Low	Moderate	Relatively high
Large search space	Limited	Suitable	Well suited
Coordinated signal changes	Limited	Possible	Strong potential
Dynamic adaptation	Quick local updates	Depends on cooling schedule	Requires repeated evaluation and updating

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

(a) Why Does This Problem Require an Online Search Agent?

An online search agent makes decisions while interacting with an unknown environment. It does not require a complete map before starting its journey.

In the Mars exploration scenario:

- The rover does not have a complete map of the terrain.
- It discovers rocks, craters, and obstacles while moving.
- It collects sensor data and updates its knowledge continuously.
- It selects actions based on the information currently available.
- It can change its route when it discovers a hazard.

Example: If the rover discovers a crater blocking its planned route, it can explore another direction instead of following the original path.

Therefore, online search is more suitable than offline search, which typically plans using a known model of the environment before execution.

(b) Challenges of Partial Observability and Dynamic Environments

Partial observability means that the rover cannot observe the complete environment at once.

The major challenges are:

- Limited sensing: Sensors may detect only nearby objects.
- Unknown terrain: The rover may encounter unexpected rocks, slopes, or craters.
- Uncertain information: Sensor readings may be incomplete or inaccurate.
- Communication delays: Signals between Mars and Earth may take several minutes, preventing immediate remote control.
- Changing conditions: Dust storms or shifting environmental conditions may affect visibility and movement.
- Safety risks: Incorrect decisions may damage the rover or prevent sample collection.

To overcome these challenges, the rover must use local sensor information, maintain a record of explored areas, and replan whenever necessary.

(c) How the Rover Builds and Updates Its Internal Model

The rover maintains an internal model, which represents the terrain it has explored and the information it has collected.

Working process:

1. Sense: Collect data using cameras, radar, and other available sensors.
2. Identify: Detect obstacles, safe paths, slopes, and potential sample locations.
3. Map: Record explored areas and their estimated movement costs.
4. Update: Modify the map when new sensor information becomes available.
5. Plan: Select the next safe action based on the updated model.
6. Execute: Move toward the target and repeat the process.

Example: If a previously unknown rock is detected, the rover marks that location as an obstacle and calculates an alternative route.

This continuous cycle helps the rover make decisions without needing a complete map in advance.

(d) Comparison of Online Search with Uninformed and Informed Search

Feature	**Uninformed Search**	**Informed Search**	**Online Search**
Knowledge required	Initial problem description	Problem description and heuristic	Learns through interaction

Environment	Usually modeled in advance	Usually modeled in advance	May be unknown
Guidance	No heuristic required	Uses a heuristic	Uses observations and available guidance
Adaptability	Limited without replanning	Depends on replanning and model updates	Naturally suited to repeated observation and replanning
Example	Breadth-First Search	A* Search	Online exploration with sensor-based replanning

Explanation:

- Uninformed search: Explores possible paths without using estimates of how close they are to the goal.
- Informed search: Uses heuristic information to guide exploration toward promising paths.
- Online search: Chooses actions while discovering the environment and learning which paths are safe.

Online search can also use informed techniques. For example, a rover can use distance to the sample location as a heuristic while continuously updating its map.

(e) Most Suitable Approach and Justification

Recommendation: Online search with heuristic guidance and continuous replanning.

Requirement	**How the approach helps**
Uncertainty	Uses sensor readings to update knowledge
Adaptability	Changes routes when new obstacles are discovered
Safety	Avoids known hazards and evaluates alternative paths
Efficiency	Uses heuristics to prioritize promising directions
Exploration	Builds a map while moving through unknown terrain

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

(a) Formulate the Problem as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) is a problem defined by variables, domains, and constraints. The goal is to assign values to all variables while satisfying every constraint.

1. Variables

Each exam is represented by a variable.

Example: \(E_1, E_2, E_3\) represent exams for Mathematics, Physics, and Computer Science.

2. Domains

The domain contains the possible time slots and exam halls available for each exam.

Example:

- Time slots: 9 AM–12 PM, 1 PM–4 PM.
- Halls: Hall A, Hall B, Hall C.

3. Constraints

Constraint	**Explanation**
Student constraint	A student must not have two exams at the same time.
Hall constraint	The number of exams scheduled simultaneously must not exceed available hall capacity.
Subject order	Certain subjects must be scheduled before other subjects.
Faculty availability	Exams must be scheduled when the required faculty members are available.
Special accommodation	Students requiring additional time or accessible facilities must receive suitable arrangements.

Objective: Assign a time slot and hall to every exam without violating the constraints.

(b) How Backtracking Search Works

Backtracking search assigns values to variables one by one. If an assignment violates a constraint, the algorithm goes back and tries another value.

Steps:

1. Select an exam that has not yet been scheduled.

2. Choose an available time slot.
3. Assign a suitable examination hall.
4. Check whether the assignment satisfies all constraints.
5. If valid, continue with the next exam.
6. If invalid, undo the assignment and try another option.
7. Continue until all exams are scheduled successfully.

Example: If Mathematics and Physics share students, they cannot be assigned to the same time slot. If the algorithm schedules them together, it backtracks and assigns one exam to a different slot.

Advantages:

- Simple and systematic.
- Finds a valid solution when one exists, provided the search is exhaustive and constraints are modeled correctly.
- Handles multiple constraints.

Limitation: The number of possible assignments can grow exponentially as the number of exams increases.

(c) Constraint Propagation and Forward Checking

Constraint propagation reduces the possible values of variables by using the constraints already established.

Forward checking is a technique that checks the remaining domains immediately after each assignment.

Example:

Suppose Mathematics is scheduled on Monday morning. Physics has students in common with Mathematics.

- Monday morning is removed from Physics's available time slots.
- Other affected exams have their domains updated.
- If any exam has no valid time slots remaining, the algorithm backtracks immediately.

Benefits:

- Detects conflicts early.
- Reduces unnecessary search.
- Avoids exploring assignments that cannot lead to a valid timetable.
- Improves efficiency for large scheduling problems.

Forward checking can be combined with the Minimum Remaining Values (MRV) heuristic, which selects the exam with the fewest legal options first. This helps resolve the most constrained exams before they cause conflicts later.

(d) Real-World Challenges and Best Strategy for Scalability

Challenges

- A large number of exams and students increases the search space.
- Limited halls may cause scheduling conflicts.
- Faculty availability may change.
- Special accommodations require additional resources and time.
- Some subjects have strict ordering requirements.
- Last-minute cancellations require timetable updates.

Recommended Strategy

Use backtracking search with constraint propagation, forward checking, and MRV.

Technique	**Benefit**
Backtracking	Systematically searches for valid assignments
Constraint propagation	Reduces domains using known constraints
Forward checking	Detects conflicts early
MRV heuristic	Selects the most constrained exam first
Degree heuristic	Prioritizes exams that constrain many other exams

For very large universities, the system can also use decomposition to schedule groups of exams or departments separately, followed by conflict checking and coordination.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

(a) How Minimax Models the Problem

Minimax is an adversarial search algorithm used in games where two players have opposing objectives. It assumes that both players make the best possible decisions.

- MAX player: The AI tries to maximize its score.
- MIN player: The human opponent tries to minimize the AI's score.
- Game tree: Represents possible moves and resulting game states.
- Utility value: Represents the outcome of a terminal state.

Working:

1. Generate possible moves from the current game state.
2. Expand the game tree to a specified depth.
3. Evaluate terminal or cutoff states.
4. MAX selects the highest-value move.
5. MIN selects the lowest-value move.
6. The AI chooses the best move assuming the opponent plays optimally.

Example: If the AI can attack, defend, or collect resources, Minimax evaluates the possible opponent responses and chooses the move with the best worst-case outcome.

(b) Computational Challenges of Large Game Trees

The game tree becomes extremely large because every move can produce many possible future states.

If b is the average number of possible moves and d is the search depth, the number of nodes is approximately:

$$ N=O(b^d) $$

For example, if each state has 10 possible moves and the AI searches 5 levels deep, it may examine up to $10^5=100,000$ nodes.

Major challenges:

- High computation time.
- Large memory requirements.
- Strict real-time decision deadlines.
- Complex interactions between units and resources.
- Uncertainty about the opponent's future moves.

Therefore, exploring every possible move is often impractical.

(c) How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

- Alpha (α): The best value MAX can guarantee so far.
- Beta (β): The best value MIN can guarantee so far.
- Pruning condition: Stop exploring a branch when $\alpha \geq \beta$.

Example

Suppose MAX is choosing between two moves, A and B.

1. Move A has possible outcomes of 5 and 6. MAX can guarantee a value of 5, so $\alpha=5$.
2. The algorithm explores move B, which is controlled by MIN.
3. The first outcome under B has a value of 4, so $\beta=4$.
4. Since $\beta \leq \alpha$, MAX already has a better option with A.
5. The remaining branches under B can be pruned because MIN can force an outcome no better than 4.

Thus, the algorithm avoids unnecessary evaluations.

Advantages:

- Produces the same optimal move as Minimax when searching the same tree.
- Evaluates fewer nodes.
- Enables deeper searches within the same time limit.
- Works especially well when promising moves are examined first.

With good move ordering, the effective search complexity can improve from approximately $O(b^d)$ to $O(b^{d/2})$ in favorable conditions.

(d) Design an Evaluation Function

An evaluation function estimates how favorable a non-terminal game state is for the AI.

One possible function is:

$$\begin{aligned} Eval(s)={}&w_1M+w_2R+w_3T\\ &+w_4P-w_5D \end{aligned}$$

Where:

- M = military strength advantage.
- R = resource advantage.
- T = territory controlled.
- P = progress toward the game objective.
- D = damage or threats to the AI's own units and base.
- $w_1,\ldots,w_5$ = weights representing the importance of each factor.

Justification:

- Military strength: Helps the AI estimate its combat advantage.
- Resources: Measures its ability to build units and maintain its economy.
- Territory: Rewards control of strategically important areas.
- Objective progress: Encourages actions that lead toward victory.
- Defensive risk: Penalizes vulnerable units and exposed bases.

The weights should be tuned through testing and gameplay simulations. Terminal states should receive very large positive values for a win and very large negative values for a loss.

(e) Recommended Strategy for Real-Time Performance

Recommendation: Minimax with Alpha-Beta pruning, iterative deepening, and a heuristic evaluation function.

Technique	**Purpose**
Alpha-Beta pruning	Eliminates unnecessary branches
Iterative deepening	Searches progressively deeper while tracking the best move
Move ordering	Examines promising moves first to improve pruning
Evaluation function	Estimates the quality of non-terminal states
Time limit	Stops the search before the decision deadline
Transposition table	Reuses results for previously evaluated game states